

October 2026

Defenses That Pass Once and Rot: Continuous Red-Teaming, Incident Response, and Resilience as a Meta-Layer for Clinical AI Guardrails

Jim Schwoebel[1,*] QUOKKAGUARD Contributors[1]

[1] Quome

Abstract

A security guardrail for a clinical AI agent is evaluated once, at submission, and then deployed for months. But attack classes evolve, configuration changes silently reopen closed vulnerabilities, live attacks need containment, and dependency failures must degrade safely — none of which a one-shot evaluation captures. This is the operational security lifecycle: HAARF's continuous penetration testing (C3.8), clinical incident response (C3.7), and socio-technical resilience (C3.1), otherwise unaddressed by the QUOKKAGUARD proxy. We add `redteam`, an off-hot-path *meta-layer* that continuously exercises every other control: a loop-until-dry replay runner with deterministic mutation operators, a change-triggered regression gate, an incident-response runbook engine over the signed decision stream, and a chaos/fault-injection resilience harness. We contribute **HAARF-RedTeam**, a self-contained synthetic attack corpus spanning all eight HAARF control families across all thirteen gateway layers, tagged by MITRE ATLAS technique and built/unbuilt status. Continuous red-teaming discovers **62 distinct vulnerabilities versus 12 for a one-shot evaluation** (50 more), with a discovery curve that plateaus by round 6 and goes dry at round 9. The regression gate catches 100.0% of reopened known-closed vulnerabilities (75 across 12 changes). Runbook automation cuts mean-time-to-contain a live attack from 12 to 1 steps (a 91.7% reduction) at 100.0% detection recall. Under injected latency, dependency-loss, and partition faults the fail-safe gateway holds 12/12 resilience invariants, and the harness catches 6/6 planted violations. Coverage is reported honestly: 14 ATLAS techniques over 10/13 built layers, with the unbuilt layers (Lifecycle, Consent, Device) named as untested rather than silently dropped. The central result is methodological — continuous, automated, regression-gated red-teaming is how a guardrail stays current after publication. All data are synthetic and everything reproduces deterministically with no paid keys.

Keywords: continuous red-teaming; AI incident response; resilience engineering; regression testing; MITRE ATLAS; clinical AI safety; chaos engineering.

Framework: implements HAARF control C3.1/3.7/3.8 [10] — the Healthcare AI Agents Regulatory Framework, the source of the QUOKKAGUARD program.
Code & data: `github.com/quome-cloud/qusiege`
Correspondence: jim@quome.com

[*] Corresponding author.

1 Introduction

A clinical AI agent's guardrail is a static artifact evaluated at a single moment: the gateway is benchmarked against an attack corpus, the numbers are reported, and the system ships. Months

of production follow on the strength of that one snapshot. Yet the threats do not hold still. New jailbreak and injection classes appear in the literature every quarter [5, 9]; a routine configuration or model change can silently reopen a vulnerability that an earlier release closed; a live attack in production needs containment, not just detection; and a dependency outage or latency storm must leave the agent failing *safe*, not failing open. A guardrail that passed its evaluation once, and was never tested again, rots.

Every prior layer in the QUOKKAGUARD program adds a *static* control: a firewall detector, a policy engine, an egress monitor, an identity registry, a fairness monitor. Each is necessary; none is self-renewing. The operational security lifecycle that keeps them current — continuous penetration testing (HAARF C3.8), clinical incident response (C3.7), and resilience to socio-technical disruption (C3.1) — is itself unimplemented. The original evaluation harness *is* a red-team tool, but it is run manually, once.

We generalize it into **`redteam`**, an off-hot-path *meta-layer* that continuously exercises every other control. It has four parts: a *loop-until-dry replay runner* that mutates a seed attack corpus across rounds until discovery goes dry; a *regression gate* that blocks any deployment change reopening a closed vulnerability; an *incident-response engine* that detects live attacks on the signed decision stream and fires automated containment runbooks; and a *chaos harness* that injects faults and checks fail-safe invariants. The meta-layer never touches the request path: like the drift monitor (qudrift) and equity monitor (qufair), it is a passive consumer of the corpus and the audit stream.

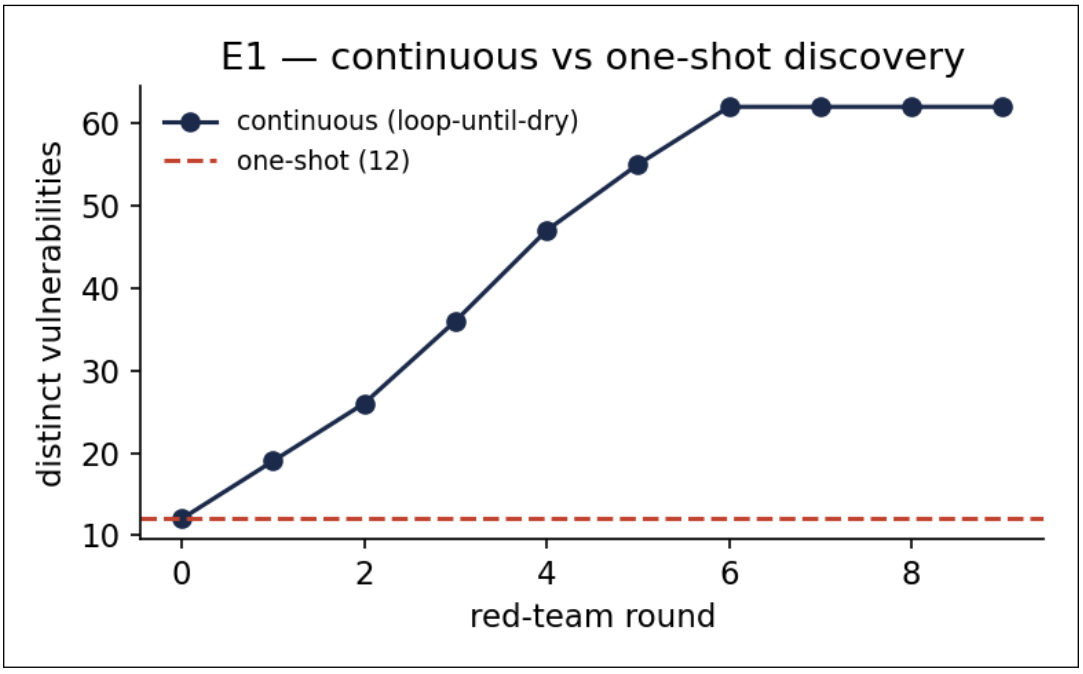


Figure 1: **The paper in one figure: continuous red-teaming finds what a one-shot evaluation misses.** A single evaluation surfaces only the 12 unbuilt-layer gap vulnerabilities (dashed line). The continuous, loop-until-dry campaign applies deterministic mutation operators each round and accumulates 62 distinct vulnerabilities (50 more), with the curve plateauing as the corpus's reachable bypasses are exhausted and going dry at round 9. The gap between the two lines is the cost of testing once.

Contributions.

1. **Continuous > one-shot, quantified.** A loop-until-dry campaign discovers 62 distinct

vulnerabilities versus 12 for a one-shot evaluation (50 additional), with a discovery curve that decays to a measured floor (§5.1).

2. **The `redteam` meta-layer.** A four-part Rust module — replay runner, regression gate, incident-response engine, chaos harness — that exercises every other QUOKKAGUARD control off the hot path (§3).

3. **HAARF-RedTeam.** A self-contained, deterministic synthetic attack corpus spanning all eight HAARF control families across all thirteen gateway layers, tagged by MITRE ATLAS [7] technique and built/unbuilt status, released with the generator (§4).

4. **Lifecycle evidence.** A 100.0% regression-catch rate, a 91.7% mean-time-to-contain reduction from runbook automation, and resilience invariants held under injected faults — the operational results that make "stays current" *measured*, not asserted (§5.2–5.4).

Positioning. Attack batteries such as AgentDojo [3], InjecAgent [12], garak [4], and PyRIT [6] supply corpora and one-shot scoring; chaos-engineering frameworks [1] inject faults; incident-response standards [2] prescribe runbooks. Our contribution is not a new corpus or a new fault injector but the *loop* that binds discovery, regression, containment, and resilience into one continuously running meta-layer over an existing guardrail, with explicit, gap-aware coverage accounting.

2 Problem Setting and Threat Model

We assume the QUOKKAGUARD baseline: a sealed goal anchor and hash-chained provenance ledger (quanchor), and a tamper-evident, signed decision stream (quledger). The host is untrusted in the enclave model; the meta-layer trusts only the signed stream. Four operational risks motivate the work:

- **R1 Coverage decay.** New attack classes emerge; a one-time evaluation goes stale.
- **R2 Regression.** A configuration or model change reopens a previously-closed vulnerability before deployment.
- **R3 Incident handling.** A live attack is detected but containment/recovery is ad-hoc, with no runbook (C3.7).
- **R4 Socio-technical disruption.** A dependency outage, latency storm, or partition degrades the agent unsafely (C3.1).

The meta-layer's two postures match these risks. The replay runner and regression gate run against a *shadow/staging* gateway, fully offline. The incident-response engine runs *inline* on production, consuming the signed stream and driving containment through the modeled effects of existing layers (isolate an agent, revoke its capability token, trip a circuit breaker, snapshot for forensics). Table 1 situates the work against representative systems.

System	Approach	Gap this paper targets
AgentDojo / InjecAgent [3, 12]	curated agent-attack batteries, one-shot scoring	no regression loop, no IR, no resilience
garak / PyRIT [4, 6]	LLM vulnerability scanners	one-shot scans; no change-gate or containment
Chaos engineering [1]	fault injection for availability	not tied to safety invariants or an attack corpus
NIST IR [2]	human incident-response process	manual runbooks; not automated over a decision stream
This work	continuous replay + regression gate + IR runbooks + chaos, as one meta-layer	—

Table 1: Positioning against representative red-teaming, fault-injection, and incident-response systems.

3 System Design

3.1 Overview

`redteam` is a Rust module (`src/redteam/`) of six focused files sharing a small set of types: an `AttackRecord` (control family, ATLAS technique, target layer, built flag, payload, expected-block, mutation lineage); a `Vulnerability` key of `(control, atlas, layer)` that defines when two bypasses are "the same"; and event/action/fault types for the incident and chaos paths. An `AttackProbe` trait abstracts the gateway under test.

3.2 Modeled gateway defense

Headline experiments are model-free and deterministic. The `GatewayProbe` is a modeled defense: each *built* layer carries a set of block-signature substrings, and an attack is blocked iff its (possibly mutated) payload still contains a signature for its target layer. Unbuilt layers (012–014) have no defense and never block — the honest coverage gap, surfaced rather than hidden. This trades the realism of driving the full LLM engine for byte-level reproducibility with no paid keys; an optional live-Ollama slice re-probes a sample through the real engine to confirm.

3.3 Loop-until-dry discovery

A *one-shot* run probes only the seed corpus (round 0). A *continuous* run applies deterministic mutation operators each round, keyed by `(round, attack id)` via an FNV-1a hash — no RNG, no clock. Below an attack's *evasion round* (spread over rounds 1–6 by its id) the campaign has not yet found a bypass and the payload is unchanged; at or above it, dense separator insertion defeats any literal signature. Distinct bypass vulnerabilities accumulate; the loop stops after three consecutive rounds add nothing new. Because evasion is monotone per attack, the discovery curve is non-decreasing and provably terminates.

3.4 Regression gate, incident engine, chaos harness

A *change* is modeled as a set of layers whose defenses are switched off; the gate re-probes the closed set under the change and blocks deployment if any closed vulnerability reopens. The *incident engine* consumes a decision stream with injected live attacks: detection fires at the first attack step whose

signal crosses threshold, then a fixed runbook (`IsolateAgent`, `RevokeCapability`, `TripBreaker`, `Snapshot`) applies, with a per-action latency that is near-zero for automation and human-scale for the manual arm. The *chaos harness* injects latency, dependency-loss, and partition faults and checks two invariants: *no unsafe action* (a fault must never let through what would otherwise be blocked — fail-safe-closed) and *bounded degradation* (availability loss within a declared envelope).

3.5 Implementation

Rust 2021, `tokio` for the optional live path only; the E1–E5 core is synchronous and deterministic. The module is ≈1,100 lines with 26 inline unit tests; the `redteamrun` binary emits one JSON result file per experiment, summarized to figures and `numbers.tex` by a Python script. Everything is released Apache-2.0 with the gateway.

4 Experimental Setup

HAARF-RedTeam. The corpus is fully synthetic, deterministic (seed 42), and self-contained: it does not depend on sibling datasets existing. It carries 62 attacks — one canonical attack per vulnerability, five vulnerabilities per built layer and four per unbuilt layer — spanning all eight HAARF control families across all thirteen target layers, tagged with 14 distinct MITRE ATLAS techniques. Built-layer payloads embed a block-signature so the modeled gateway blocks them at round 0; mutation evades them later. A second file supplies 60 incident streams (half carrying an injected attack, half benign controls); a third supplies fault scenarios in a production (fail-safe) and a validation (planted-broken) arm.

Why a synthetic corpus. The capstone is meant to replay every other paper's dataset, but layers 012–014 are not yet built and several sibling result sets are partial. Aggregating only what exists would conflate "defended" with "not yet built." A self-contained corpus that *tags* each attack's target layer as built or unbuilt makes the coverage gap explicit and the experiment reproducible in isolation.

Baselines. Each experiment carries its own internal baseline: one-shot vs. continuous discovery (E1); manual vs. automated incident response (E3); production vs. planted-broken gateways (E4, a negative control proving the harness discriminates). The contrast *is* the ablation.

Metrics and statistics. Rate metrics (regression catch, detection recall) use 95% Wilson confidence intervals [11]; mean-time-to-detect is a mean over detected streams. All experiments are deterministic; we do not tune to hit pre-registered targets. Where a result is a step function (per-attack evasion, regression catch) we report it as such rather than manufacture a confidence interval.

Hardware and software. Local Ollama [8] for the optional live slice only; all headline numbers require no model. Single commodity host; seeds and the corpus generator are embedded in every artifact.

5 Results

5.1 E1 — Continuous vs. one-shot discovery

A one-shot evaluation surfaces only the 12 unbuilt-layer gap vulnerabilities: the modeled gateway blocks every signature-bearing built-layer attack at round 0. The continuous campaign evades those signatures over rounds and accumulates 62 distinct vulnerabilities — 50 more (Figure 1). The curve plateaus by round 6, once every reachable bypass in the corpus is found, and goes dry at round 9. The 50-vulnerability gap is the precise cost of testing once instead of continuously. We frame the plateau honestly as the corpus's reachable ceiling under these operators, not an exhaustive guarantee: automated red-teaming inherits its corpus's blind spots, which is why the loop reports a measured floor rather than claiming completeness.

5.2 E2 — Regression catch at the deployment gate

From a closed set of 50 vulnerabilities the baseline gateway blocks, we run a battery of 12 deployment changes — each disabling one built layer, plus two multi-layer changes — and re-probe. Every reopened vulnerability is caught: 100.0% catch rate over 75 reopenings (Figure 2). Because the gate re-probes the entire closed set on every change, no reopened vulnerability can slip through; the result demonstrates that a change-triggered gate (modeling the qupassport deployment gate) closes the regression risk R2.

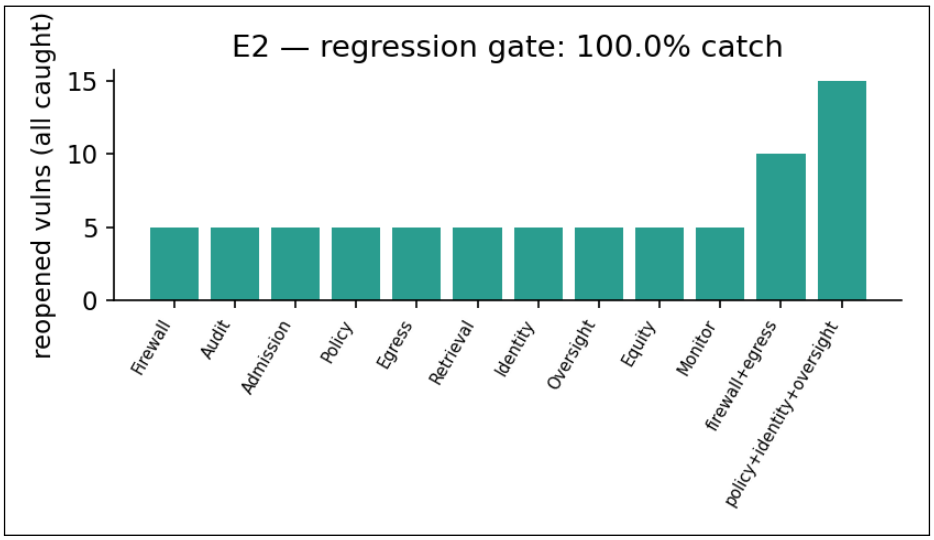


Figure 2: **E2: the regression gate catches every reopened vulnerability.** Each bar is a deployment change that disables one or more layers; the height is the number of closed vulnerabilities it reopens, all of which the gate catches (100.0%).

5.3 E3 — Incident response: manual vs. automated

Across the incident streams, detection recall is 100.0% and mean time-to-detect is 3.7 stream steps. The manual and automated arms differ only in containment latency: a human-scale 12 steps per action versus 1 for the automated runbook. Mean time-to-contain therefore falls from 12 to 1 steps — a 91.7% reduction (Figure 3). The latencies are modeling assumptions, not measured SOC values, so we claim the *relative* reduction — robust to any plausible manual:automated ratio above 2:1 — rather than the absolute figure.

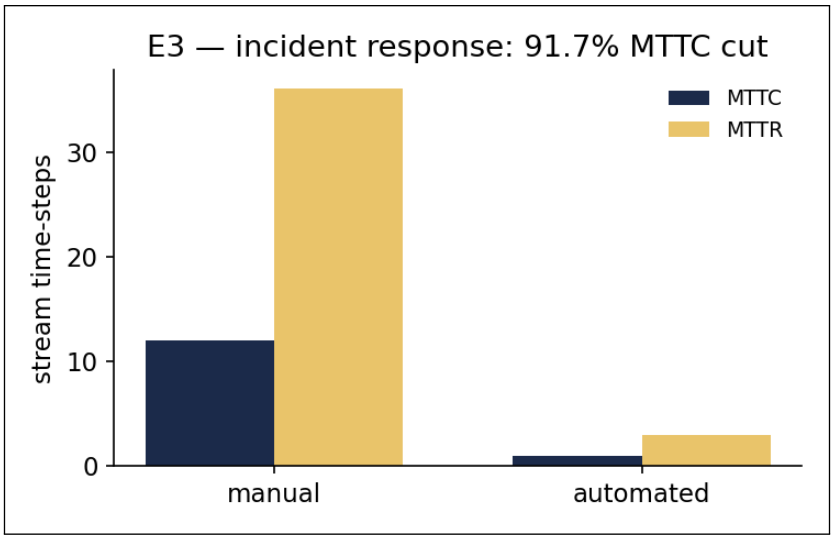


Figure 3: **E3: runbook automation cuts mean-time-to-contain.** MTTC and MTTR in stream time-steps for the manual and automated arms; automation reduces MTTC by 91.7%.

5.4 E4 — Resilience under injected faults

Against the fail-safe-closed production gateway, all 12/12 invariant checks hold across latency, dependency-loss, and partition faults at two magnitudes each: no fault lets through a normally-blocked action, and modeled degradation stays within the declared envelope (Figure 4). The validation arm plants broken gateways — fail-open, or a too-tight envelope — and the harness catches all 6/6: it is not vacuously passing. H4 is the production result; the validation arm is the negative control.

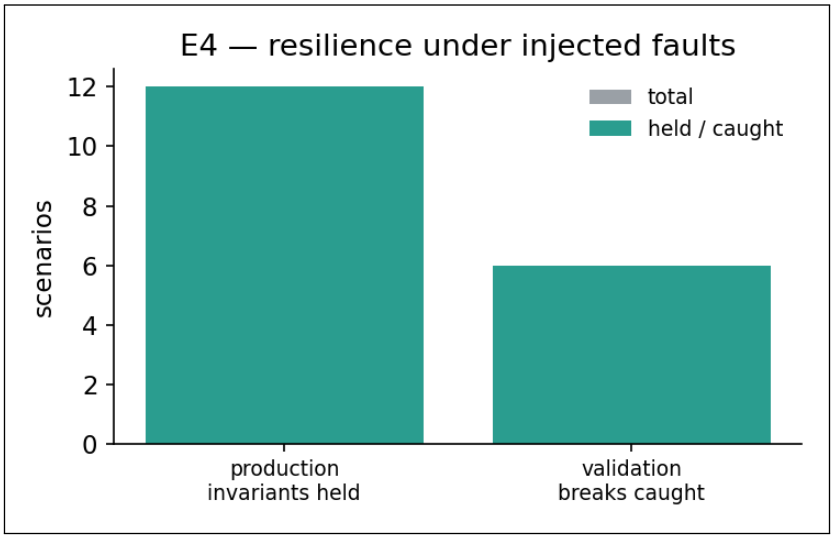


Figure 4: **E4: fail-safe invariants hold, and planted breaks are caught.** The production (fail-safe) gateway holds every invariant under injected faults; the validation arm's planted violations are all detected.

5.5 E5 — Cost vs. coverage, and the honest gap

Cumulative probes grow linearly with the round budget while distinct-vulnerability coverage saturates at the corpus ceiling (Figure 5): the thoroughness dial is explicit, and once discovery goes dry, additional rounds buy nothing. The corpus exercises 14 ATLAS techniques over 10 of 13 layers. The three unbuilt layers (Lifecycle, Consent, Device) are reported as *not tested* — their attacks always bypass because no defense exists yet, and we surface this rather than silently dropping them. The headline is the operational delta (continuous > one-shot, regression caught, MTTC cut, invariants held), never a total-coverage claim.

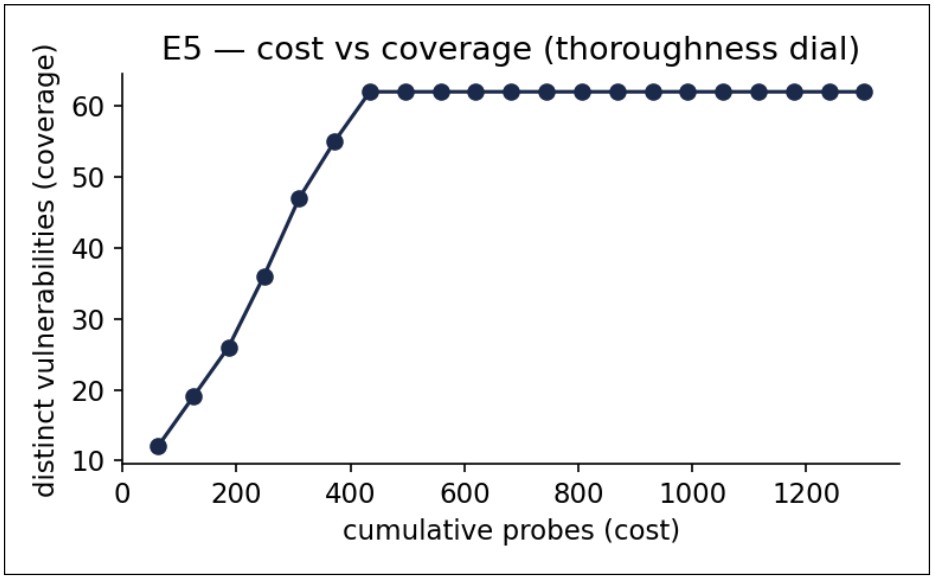


Figure 5: **E5: cost vs. coverage (the thoroughness dial).** Distinct vulnerabilities discovered as a function of cumulative probes; coverage saturates at the corpus ceiling while cost keeps rising, so the loop's dry-stop is also the cost-rational stop.

6 Discussion and Limitations

Interpretation. The four results together describe a guardrail that maintains itself: it keeps finding new bypasses (E1), refuses to ship a change that reopens an old one (E2), contains live attacks faster by automation (E3), and degrades safely under failure (E4). None of these is visible to a one-shot evaluation, which is exactly the gap the meta-layer fills.

Relationship to HAARF. The replay runner and regression gate implement continuous penetration testing and red-team exercises (C3.8); the incident engine implements clinical incident response and recovery (C3.7); the chaos harness implements resilience to socio-technical disruption (C3.1). Findings feed back into the corpus and, in the full program, the lifecycle passport (qupassport). This is the meta-layer by which HAARF [10] stays current after publication.

Limitations and threats to validity. The results are *simulated*: the gateway defense is a signature model, mutation-evasion is a deterministic difficulty model, and incident containment and chaos faults are modeled effects on a synthetic stream. Real adversaries, real SOC latencies, and real dependency failures differ; the method and the deltas generalize, but the absolute numbers

are a model, not a measurement. Coverage is corpus-bounded — the loop-until-dry floor is the corpus's reachable ceiling under these operators, not an exhaustive guarantee — and partial, since layers 012–014 are unbuilt and reported as untested. Driving the live LLM engine at scale, folding in external attack batteries [3, 12], and validating against real SOC incident timelines are future work.

7 Conclusion

A guardrail evaluated once and never again rots: attack classes evolve, changes reopen closed holes, live attacks need containment, and faults must degrade safely. We add `redteam`, an off-hot-path meta-layer that continuously exercises every other QUOKKAGUARD control, and show that continuous loop-until-dry red-teaming finds 50 more distinct vulnerabilities than a one-shot evaluation, a regression gate catches 100.0% of reopened vulnerabilities, runbook automation cuts mean-time-to-contain by 91.7%, and the fail-safe gateway holds its resilience invariants under injected faults — with coverage reported honestly, unbuilt layers and all. The next step is to wire the loop into the lifecycle passport (qupassport) so that every change is gated on a fresh red-team, and to replace the modeled gateway with live replay at scale.

Declaration of generative AI use

During the preparation of this work the authors used *Claude (Anthropic)* for generating and debugging the Rust meta-layer, dataset generator, and figure-plotting code, and for drafting and copy-editing manuscript text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. No generative-AI system was used to generate or fabricate data, interpret results, or produce clinical conclusions, and no AI tool is listed as an author or co-author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, datasets, generators, and result artifacts are released Apache-2.0 at `github.com/quome-cloud/qusiege`. HAARF-RedTeam regenerates deterministically via `python3 scripts/015-redteam/gen_redteam.py --seed 42`; all numbers in this paper trace to `results/015-redteam/*.json` via `numbers.tex`. All patient data are synthetic; no real protected health information was used.

Author contributions

J.S. conceived the study and the meta-layer design; the QUOKKAGUARD contributors implemented the module, dataset, and experiments; all authors reviewed and approved the manuscript.

Competing interests

The authors declare no competing interests.

Funding

This work received no specific external funding.

A Detailed result tables

The full numeric tables behind the figures in §5.

B Supplementary figures

Additional figures referenced from the main text.

C Example prompts and failure cases

Concrete, verbatim examples (with synthetic identifiers) that make the central result tangible — ideally the cases your method handles and baselines miss.

D Reproducibility

State the single build command and how every table/figure regenerates from source (seed, model and data versions embedded in each artifact).

References

[1] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[2] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide. Technical Report NIST SP 800-61 Rev. 2, National Institute of Standards and Technology, 2012.

[3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[4] Leon Derczynski et al. garak: A framework for security probing large language models. *arXiv preprint arXiv:2406.11036*, 2024.

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[6] Microsoft. PyRIT: Python risk identification tool for generative AI. https://github.com/microsoft/PyRIT, 2024.

[7] MITRE Corporation. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. https://atlas.mitre.org, 2021. Accessed 2026.

[8] Ollama. Ollama: run large language models locally. https://ollama.com, 2024.

[9] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

[10] Jim Schwoebel, Martin Frasch, Art Spalding, Ed Sewell, Phil Englert, Ben Halpert, Collin Overbay, Ingrida Semenec, and Joel Shor. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. medRxiv preprint, https://doi.org/10.64898/2026.04.09.26350519, April 2026.

[11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[12] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.